

HW7_Run

March 19, 2026

1 Homework 7 : GANs

In this assignment, you will learn to write an advanced PyTorch implementation concept commonly used in a complex deep learning pipeline by using GAN as a learning example.

You will also start working with more complex architectures (upsampling) and writing style (complex modules such as modulelist) for pytorch.

Every TODO is weighted equally. Optional TODO is half of a regular TODO.

2 Part 1 : WGAN-GP reimplementation

In this section, you are going to reimplement WGAN-GP (<https://arxiv.org/pdf/1704.00028.pdf>) based on the pseudocode provided in the paper to generate MNIST digit characters. Some parts are intentionally modified to discourage straight copy-pasting from public repositories.

The pseudocode could be organized into two main parts: discriminator optimization in line 2-10, and generator optimization in line 11-12.

The discriminator part consists of four steps: - Line 4: data, and noise sampling with a batch size of m - Line 5-7: discriminator loss calculation - Line 9: discriminator update - Repeat line 4-9 for n_{critic} steps

After the discriminator is updated, the generator is then updated by performing two steps: - Line 11: noise sampling - Line 12: generator loss calculation and update

This part is divided into four subsections: network initialization, hyperparameter initialization, data preparation, and training loop. The detail for each part will be explained in the subsections.

2.1 Downloading MNIST dataset

The MNIST dataset contains 60,000 training digit character image (0-9) at 28x28 resolution that are normalized to $[0, 1]$. Given the training images, your task is to generate new training images using WGAN-GP by learning from the training distribution.

```
[4]: import torchvision.datasets as datasets
import numpy as np
```

```
mnist_trainset = datasets.MNIST(root='./data', train=True, download=True,
    ↪transform=None)
trainX = np.array(mnist_trainset.data[... , None]).transpose(0, 3, 1, 2) / 255
print("Dataset size : ", trainX.shape)
```

Dataset size : (60000, 1, 28, 28)

C:\Users\Faith\AppData\Local\Temp\ipykernel_13848\660608400.py:5:

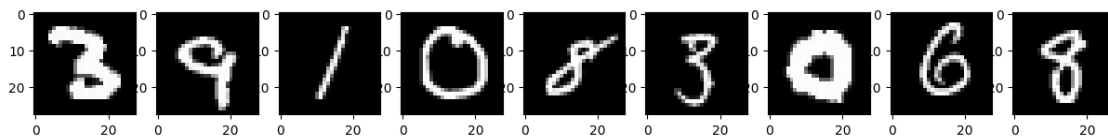
DeprecationWarning: __array__ implementation doesn't accept a copy keyword, so passing copy=False failed. __array__ must implement 'dtype' and 'copy' keyword arguments. To learn more, see the migration guide

https://numpy.org/devdocs/numpy_2_0_migration_guide.html#adapting-to-changes-in-the-copy-keyword

```
trainX = np.array(mnist_trainset.data[... , None]).transpose(0, 3, 1, 2) / 255
```

2.2 Dataset Visualization

```
[5]: import matplotlib.pyplot as plt
import numpy as np
plt.figure(figsize = (15,75))
for i in range(9):
    plt.subplot( int('19{}'.format(i+1)) )
    plt.imshow( trainX[np.random.randint(len(trainX))].transpose((1, 2, 0))[... ,
    ↪0] , cmap = 'gray' )
plt.show()
```



2.3 Generator and Discriminator network

Before training, the deep learning networks have to be initialized first. Therefore, in this part, you are going to write a generator and discriminator network based on the description provided below.

The description of the discriminator network is shown in the Table below.

The network also has some specific requirements: - ConvBlock is a Convolution-ReLU layer - All ReLUs in the encoder are leaky, with a slope of 0.1

The description of the generator network is shown in the Table below.

The network also has some specific requirements: - ConvBlock is a ConvTranspose-BatchNorm-ReLU layer - Downsampling method is bilinear interpolation (torch.nn.Upsample or torch.nn.functional.interpolate)

TODO 1: Implement a discriminator network.

TODO 2: Implement a generator network.

```
[6]: import torch
import torch.nn.functional as F
from torch import nn
from torchvision import transforms

class Discriminator(nn.Module):
    ##TODO1 implement the discriminator (critic)
    def __init__(self):
        super().__init__()
        self.conv = nn.Sequential(
            nn.Conv2d(1, 128, kernel_size=5, stride=2, padding=2),
            nn.LeakyReLU(0.1),
            nn.Conv2d(128, 256, kernel_size=5, stride=2, padding=2),
            nn.LeakyReLU(0.1),
            nn.Conv2d(256, 512, kernel_size=5, stride=2, padding=2),
            nn.LeakyReLU(0.1),
        )
        fc_in = 4 * 4 * 512
        self.fc = nn.Linear(fc_in, 1)

    def forward(self, x):
        feature = self.conv(x)
        flattened_feature = feature.flatten(start_dim=1)
        output = self.fc(flattened_feature)
        return output

class Generator(nn.Module):
    ##TODO2 implement the generator (actor)
    # z is the latent inputs
    def __init__(self):
        super().__init__()
        self.fc = nn.Linear(128, 512 * 4 * 4)

        self.upsampling = nn.Sequential(
            nn.ConvTranspose2d(512, 256, kernel_size=5, stride=2, padding=2, ↵
↵output_padding=1),
            nn.BatchNorm2d(256),
            nn.ReLU(),
            nn.ConvTranspose2d(256, 128, kernel_size=5, stride=2, padding=2, ↵
↵output_padding=1),
```

```

        nn.BatchNorm2d(128),
        nn.ReLU(),
        nn.ConvTranspose2d(128, 1, kernel_size=5, stride=2, padding=2,
↪output_padding=1),
        nn.Sigmoid(),
    )

    def forward(self, x):
        feature_in = self.fc(x)
        feature_in_reshape = feature_in.reshape(-1, 512, 4, 4)
        upsampled = self.upsampling(feature_in_reshape)
        output = F.interpolate(upsampled, (28, 28), mode="bilinear",
↪align_corners=False)
        return output

discriminator = Discriminator().cuda()
generator = Generator().cuda()

```

2.4 Network verification

TODO 3: What is the input and output shape of the generator and discriminator network? Verify that the implemented networks are the same as the answer you have provided.

```

[7]: from torchinfo import summary

print(summary(Discriminator(), input_size=(32, 1, 28, 28)))

```

```

=====
=====
Layer (type:depth-idx)                   Output Shape              Param #
=====
=====
Discriminator                           [32, 1]                  --
  Sequential: 1-1                       [32, 512, 4, 4]          --
    Conv2d: 2-1                         [32, 128, 14, 14]        3,328
    LeakyReLU: 2-2                     [32, 128, 14, 14]        --
    Conv2d: 2-3                        [32, 256, 7, 7]          819,456
    LeakyReLU: 2-4                     [32, 256, 7, 7]          --
    Conv2d: 2-5                        [32, 512, 4, 4]          3,277,312
    LeakyReLU: 2-6                     [32, 512, 4, 4]          --
  Linear: 1-2                           [32, 1]                  8,193
=====
=====
Total params: 4,108,289
Trainable params: 4,108,289
Non-trainable params: 0
Total mult-adds (G): 2.98
=====

```

```

=====
Input size (MB): 0.10
Forward/backward pass size (MB): 11.73
Params size (MB): 16.43
Estimated Total Size (MB): 28.26
=====
=====

```

```

[8]: from torchinfo import summary

print(summary(Generator(), input_size=(32, 128)))

```

```

=====
=====
Layer (type:depth-idx)                Output Shape                Param #
=====
Generator                             [32, 1, 28, 28]            --
  Linear: 1-1                          [32, 8192]                 1,056,768
  Sequential: 1-2                      [32, 1, 32, 32]            --
    ConvTranspose2d: 2-1               [32, 256, 8, 8]           3,277,056
    BatchNorm2d: 2-2                  [32, 256, 8, 8]           512
    ReLU: 2-3                          [32, 256, 8, 8]            --
    ConvTranspose2d: 2-4               [32, 128, 16, 16]          819,328
    BatchNorm2d: 2-5                  [32, 128, 16, 16]          256
    ReLU: 2-6                          [32, 128, 16, 16]          --
    ConvTranspose2d: 2-7               [32, 1, 32, 32]            3,201
    Sigmoid: 2-8                      [32, 1, 32, 32]            --
=====
=====
Total params: 5,157,121
Trainable params: 5,157,121
Non-trainable params: 0
Total mult-adds (G): 13.56
=====
=====
Input size (MB): 0.02
Forward/backward pass size (MB): 27.53
Params size (MB): 20.63
Estimated Total Size (MB): 48.17
=====
=====

```

```

[9]: device = "cuda" if torch.cuda.is_available() else "cpu"

test_dis = Discriminator().to(device)
test_gen = Generator().to(device)

```

```

i_dis = torch.zeros([32, 1, 28, 28]).to(device) # (batch, channel, w, h)
i_gen = torch.zeros([32, 128]).to(device) # (batch, latent)

o_dis = test_dis(i_dis)
o_gen = test_gen(i_gen)

print(f"Input of Discriminator: {i_dis.shape}")
print(f"Output of Discriminator: {o_dis.shape}")
print('-' * 20)
print(f"Input of Generator: {i_gen.shape}")
print(f"Output of Generator: {o_gen.shape}")

```

```

Input of Discriminator: torch.Size([32, 1, 28, 28])
Output of Discriminator: torch.Size([32, 1])
-----
Input of Generator: torch.Size([32, 128])
Output of Generator: torch.Size([32, 1, 28, 28])

```

2.5 Parameter Initialization

After the network is initialized, we then set up training hyperparameters for the training. In this part, hyperparameters have already been partially provided in the cell below, though some of them are intentionally left missing (None). Your task is to fill the missing parameters based on the pseudocode above.

TODO4: Initialize the missing model hyperparameters and optimizers based on the pseudocode above.

Note: To hasten the training process of our toy experiment, the training step and batch size is reduced to 3000 and 32, respectively.

```

[10]: NUM_ITERATION = 3000
      BATCH_SIZE = 32
      fixed_z = torch.randn((8, 128)).cuda()
      def schedule(i):
          lr = 1e-4
          if(i > 2500): lr *= 0.1
          return lr
      losses = {'D' : [None], 'G' : [None]}

      ## TODO4 initialize missing hyperparameter and optimizer
      G_optimizer = torch.optim.Adam(generator.parameters(), betas=(0.0, 0.9), lr=0.
          ↪0.0001)
      D_optimizer = torch.optim.Adam(discriminator.parameters(), betas=(0.0, 0.9),
          ↪lr=0.0001)
      GP_lambda = 10
      n_critic = 5

```

2.6 Data preparation

TODO 5: Create a dataloader that could generate the data in line 4. The dataloader should return $\mathbf{x}, \mathbf{z}$, with a batch size of `BATCH_SIZE`

```
[11]: # TODO5 implement dataloader
from torch.utils.data import Dataset, DataLoader

class WGANDataset(Dataset):
    def __init__(self, x):
        super(WGANDataset, self).__init__()
        self.x = torch.from_numpy(x).float()

    def __len__(self):
        return self.x.shape[0]

    def __getitem__(self, idx):
        x = self.x[idx]
        z = torch.randn(128) # p is standard gaussian of size 128
        e = torch.rand(1) # random e (0,1) shape 1
        return x, z, e

dataset = WGANDataset(trainX)

data_loader = DataLoader(dataset, batch_size=BATCH_SIZE, shuffle=True)
```

2.7 Training loop

This section is the place where the training section starts. It is highly recommended that you understand the pseudocode before performing the tasks below. To train the WGAN-GP you have to perform the following tasks:

TODO6: Update the learning rate base on the provided scheduler.

TODO7: Sample the data from the dataloader (Line 4).

TODO8 : Calcualte the discriminator loss (Line 5-7). - In the line 7 you have to implement the gradient penalty term $\lambda(\|\hat{\mathbf{x}} D_w(\hat{\mathbf{x}})\|_2 - 1)^2$, which is a custom gradient. You may read <https://pytorch.org/docs/stable/generated/torch.autograd.grad.html> to find how custom gradient is implemented. - HINT: Gradient norm calculation is still part of the computation graph.

TODO9: Update the discriminator loss (Line 9).

TODO10: Calculate and update the generator loss (Line 11-12).

If your implementation is correct, the generated images should resemble an actual digit character after 500 iterations.

```
[12]: from tqdm import tqdm

loader_iterator = iter(data_loader)

for i in tqdm(range(NUM_ITERATION)):
```

```

## TODO6 update learning rate
current_lr = schedule(i)
D_optimizer.param_groups[0]['lr'] = current_lr
G_optimizer.param_groups[0]['lr'] = current_lr

for t in range(n_critic):
    ## TODO7 line 4: sample data from dataloader
    # sample only one batch size
    try:
        x, z, e = next(loader_iterator)
    except StopIteration:
        loader_iterator = iter(data_loader)
        x, z, e = next(loader_iterator)

    x, z, e = x.to(device), z.to(device), e.to(device)

    ## TODO8 line5-7 : calculate discriminator loss
    # x: (batch, 1, 28, 28)
    # e: (batch, 1)
    x_ = generator(z)
    e = e.reshape(-1, 1, 1, 1)
    x_generated = e * x + (1 - e) * x_
    discriminate_x = discriminator(x_generated.requires_grad_(True))

    gradients = torch.autograd.grad(
        outputs=discriminate_x,
        inputs=x_generated,
        grad_outputs=torch.ones_like(discriminate_x),
        retain_graph=True,
        create_graph=True,
    )[0].reshape(x.size(0), -1) # (batch, -1)

    gradients_norm = gradients.norm(2, dim=1)

    loss_D = discriminator(x_).mean() - discriminator(x).mean() + GP_lambda * ↪
    ((gradients_norm - 1) ** 2).mean()

    ## TODO9 : line 9 update discriminator loss
    D_optimizer.zero_grad()
    loss_D.backward()
    D_optimizer.step()

    losses['D'].append(loss_D.item())

## TODO10 : line 11-12 calculate and update the generator loss
    z = torch.randn(BATCH_SIZE, 128).to(device)
    x_generated = generator(z)

```

```
loss_G = - discriminator(x_generated).mean()
```

```
G_optimizer.zero_grad()
```

```
loss_G.backward()
```

```
G_optimizer.step()
```

```
losses['G'].append(loss_G.item())
```

Output visualization : If your reimplementation is correct, the generated images should start resembling a digit character after 500 iterations.

```
if(i % 100 == 0):
```

```
    plt.figure(figsize = (15,75))
```

```
    print(losses['D'][-1], losses['G'][-1])
```

```
    with torch.no_grad():
```

```
        res = generator(fixed_z).cpu().detach().numpy()
```

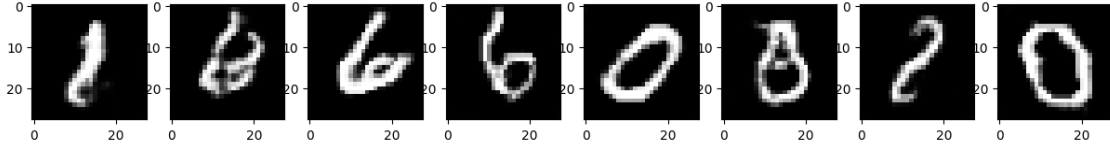
```
    for k in range(8):
```

```
        plt.subplot( int('18{}'.format(k+1)) )
```

```
        plt.imshow( res[k].transpose(1, 2, 0)[..., 0], cmap = 'gray' )
```

```
    plt.show()
```

97%| | 2899/3000 [03:13<00:06, 16.01it/s]
-1.8765220642089844 -0.870729386806488



100%| | 3000/3000 [03:19<00:00, 15.01it/s]

3 Part 2 : pix2pix reimplementation (cGAN on paired image translation)

In this exercise, we are reimplementing a paired image translation model, an application of a generative adversarial network (GAN). The model we are going to implement is pix2pix (<https://arxiv.org/pdf/1611.07004.pdf>), one of the earliest paired image translation models based on GAN. The pipeline of pix2pix is shown in the Figure below.

From the figure above, the pipeline consists of three main parts:

- 1. Generation phase : the generator G create the generated image $G(x)$ from the given input x .
- 2. Discrimination phase :
In step 2.1, the discriminator D receives an input image x and the generated image $G(x)$, then the discriminator has to learn to predict that the generated image $G(x)$ is fake.
In step 2.2, the discriminator D receives an input image x and the ground truth image y , then the discriminator has to learn to predict that the image y is real.
- 3. Refinement phase: Refine the quality of the generated image $G(x)$ by encouraging the generated image to be close to an actual image y by using L1 as an objective.

The objective of pix2pix is to train an optimal generator G^* base on the objective function :

$$G^* = \underset{G}{\operatorname{argmin}} \underset{D}{\operatorname{max}} L_{cGAN}(G, D) + \lambda L_1(G)$$

- The term $\underset{G}{\operatorname{argmin}} \underset{D}{\operatorname{max}} L_{cGAN}(G, D)$ is the objective function of the first and second step, which is a standard cGAN loss : $L_{cGAN}(G, D) = E_{x,y}[\log D(x,y)] + E_{x,z}[\log(1 - D(x, G(x,z)))]$. The noise z is embedded in the generator in the form of dropout.
- The term $L_1(G)$ is the objective fuction of the third step where $L_1(G) = E_{x,y,z}[||y - G(x, z)||_1]$

The subsections will explain the dataset and training setup of this exercise.

3.1 Get dataset

```
[13]: import urllib.request
import tarfile

url = "http://efrosgans.eecs.berkeley.edu/pix2pix/datasets/facades.tar.gz"
filename = "facades.tar.gz"

urllib.request.urlretrieve(url, filename)

with tarfile.open(filename, "r:gz") as tar:
    tar.extractall()
```

3.2 Import library

```
[14]: import cv2
import glob
import numpy as np
import torch
import torch.nn.functional as F
from torch import nn
from torchvision import transforms
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
```

3.3 Setting up facade dataset

The dataset chosen for this exercise is the CMP Facade Database which is a pair of facade images and its segmented component stored in RGB value. The objective of this exercise is to generate a facade given its simplified segmented component. Both input and output is a 256 x 256 RGB image normalized to $[-1, 1]$.

```
[15]: train = (np.array([cv2.imread(i) for i in glob.glob('facades/train/*')], dtype=np.
    ↳ np.float32) / 255).transpose((0, 3, 1, 2))
train = (train - 0.5) * 2 #shift from [0,1] to [-1, 1]
trainX = train[:, :, :, 256:]
trainY = train[:, :, :, :256]

val = (np.array([cv2.imread(i) for i in glob.glob('facades/val/*')], dtype = np.
    ↳ float32) / 255).transpose((0, 3, 1, 2))
val = (val - 0.5) * 2 #shift from [0,1] to [-1, 1]
valX = val[:, :, :, 256:]
valY = val[:, :, :, :256]

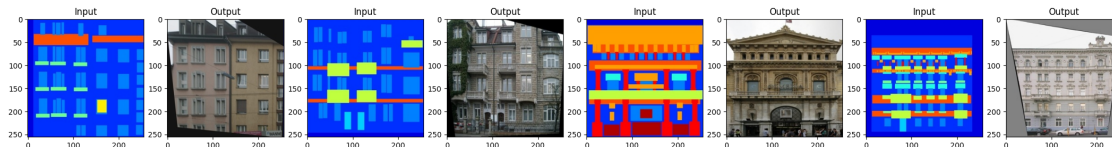
print("Input size : {}, Output size = {}".format(trainX.shape, trainY.shape))
```

Input size : (400, 3, 256, 256), Output size = (400, 3, 256, 256)

3.4 Dataset Visualization

```
[16]: import matplotlib.pyplot as plt
import numpy as np
plt.figure(figsize = (30,90))
for i in range(4):
    idx = np.random.randint(len(trainX))

    plt.subplot( int('19{}'.format(2*i+1)) )
    plt.title('Input')
    plt.imshow( (0.5 * trainX[idx].transpose((1, 2, 0)) + 0.5)[..., ::-1] , cmap='gray' )
    plt.subplot( int('19{}'.format(2*i+2)) )
    plt.title('Output')
    plt.imshow( (0.5 * trainY[idx].transpose((1, 2, 0)) + 0.5)[..., ::-1], cmap='gray' )
plt.show()
```



3.5 Note

If you have trouble understanding the instruction provided in this homework or have any ambiguity about the instruction, you could also read the appendix section (section 6.1-6.2) in the paper for a detailed explanation.

3.6 Discriminator network

In this section, we are going to implement a discriminator network of pix2pix. The description of the discriminator network is provided in the Figure below.

The network also has the following specific requirements: - All convolutions are 4×4 spatial filters - ConvBlock is a Convolution-InstanceNorm-ReLU layer - InstanceNorm is not applied to the first C64 layer - All ReLUs are leaky, with a slope of 0.2

TODO 11: Implement the discriminator network based on the description above.

TODO 12: What should be the size of the input and output of the discriminator for this task? Verify that the input and output of the implemented network are the same as the answer you have provided.

```
[17]: class Discriminator(nn.Module):
    #TODO11 implement the discriminator network
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(in_channels=6, out_channels=64, kernel_size=4,
↪stride=2, padding=1) # input as 6 channels because it takes both x and y as
↪input
        self.conv2 = nn.Conv2d(in_channels=64, out_channels=128, kernel_size=4,
↪stride=2, padding=1)
        self.in2 = nn.InstanceNorm2d(128)
        self.conv3 = nn.Conv2d(in_channels=128, out_channels=256, kernel_size=4,
↪stride=2, padding=1)
        self.in3 = nn.InstanceNorm2d(256)
        self.conv4 = nn.Conv2d(in_channels=256, out_channels=512, kernel_size=4,
↪stride=1, padding=1)
        self.in4 = nn.InstanceNorm2d(512)
        self.conv5 = nn.Conv2d(in_channels=512, out_channels=1, kernel_size=4,
↪stride=1, padding=1)

    def forward(self, x, y=None):
        if (y is not None):
            x = torch.concat([x, y], dim=1)

        feature1 = F.leaky_relu(self.conv1(x), negative_slope=0.2)
        feature2 = F.leaky_relu(self.in2(self.conv2(feature1)), negative_slope=0.
↪2)
        feature3 = F.leaky_relu(self.in3(self.conv3(feature2)), negative_slope=0.
↪2)
        feature4 = F.leaky_relu(self.in4(self.conv4(feature3)), negative_slope=0.
↪2)
        feature5 = torch.sigmoid(self.conv5(feature4))
        return feature5

discriminator = Discriminator().cuda()
```

```
[18]: #TODO12 verify the discriminator
from torchsummary import summary

summary(discriminator, input_size=(6, 256, 256))
```

```
-----
Layer (type)           Output Shape          Param #
=====
          Conv2d-1      [-1, 64, 128, 128]      6,208
          Conv2d-2      [-1, 128, 64, 64]     131,200
InstanceNorm2d-3       [-1, 128, 64, 64]         0
          Conv2d-4      [-1, 256, 32, 32]     524,544
-----
```

InstanceNorm2d-5	[-1, 256, 32, 32]	0
Conv2d-6	[-1, 512, 31, 31]	2,097,664
InstanceNorm2d-7	[-1, 512, 31, 31]	0
Conv2d-8	[-1, 1, 30, 30]	8,193

```

Total params: 2,767,809
Trainable params: 2,767,809
Non-trainable params: 0
-----
Input size (MB): 1.50
Forward/backward pass size (MB): 27.51
Params size (MB): 10.56
Estimated Total Size (MB): 39.57
-----

```

3.7 Generator network

In this section, we are going to implement a generator network of pix2pix. The generator is based on the U-NET based architecture (<https://arxiv.org/abs/1505.04597>). The Description of the generator network is provided in the Figure below.

The network also has the following specific requirements: - All convolutions are 4×4 spatial filters - ConvBlock is a Convolution-InstanceNorm-ReLU layer - ConvTBlock is a ConvolutionTranspose-InstanceNorm-DropOut-ReLU layer with a dropout rate of 50% - InstanceNorm is not applied to the first C64 layer in the encoder - All ReLUs in the encoder are leaky, with a slope of 0.2, while ReLUs in the decoder are not leaky

TODO 13: Implement the generator network based on the description above.

TODO 14: What should be the size of the input and output of the generator for this task? Verify that the input and output of the implemented network are the same as the answer you have provided.

```

[19]: #HINT : you could also put multiple layers in a single list using nn.ModuleList
class Example(nn.Module):
    def __init__(self):
        super().__init__()
        self.convs = nn.ModuleList([nn.Conv2d(25, 25, 3) for i in range(5)])
    def forward(self, x):
        for i in range(len(self.convs)):
            x = self.convs[i](x)
        return x
ex = Example().cuda()
print(ex(torch.zeros((8, 25, 32, 32)).cuda() ).shape)

```

```
torch.Size([8, 25, 22, 22])
```

```

[20]: class Generator(nn.Module):
        #TODO13 implement the generator network
        def __init__(self):
            super().__init__()
            self.convb1 = nn.Sequential(
                nn.Conv2d(in_channels=3, out_channels=64, stride=2, kernel_size=4,
                ↪padding=1),
                nn.LeakyReLU(0.2),
            )
            self.convb2 = nn.Sequential(
                nn.Conv2d(in_channels=64, out_channels=128, stride=2, kernel_size=4,
                ↪padding=1),
                nn.InstanceNorm2d(128),
                nn.LeakyReLU(0.2),
            )
            self.convb3 = nn.Sequential(
                nn.Conv2d(in_channels=128, out_channels=256, stride=2, kernel_size=4,
                ↪padding=1),
                nn.InstanceNorm2d(256),
                nn.LeakyReLU(0.2),
            )
            self.convb4 = nn.Sequential(
                nn.Conv2d(in_channels=256, out_channels=512, stride=2, kernel_size=4,
                ↪padding=1),
                nn.InstanceNorm2d(512),
                nn.LeakyReLU(0.2),
            )
            self.convb5 = nn.Sequential(
                nn.Conv2d(in_channels=512, out_channels=512, stride=2, kernel_size=4,
                ↪padding=1),
                nn.InstanceNorm2d(512),
                nn.LeakyReLU(0.2),
            )
            self.convb6 = nn.Sequential(
                nn.Conv2d(in_channels=512, out_channels=512, stride=2, kernel_size=4,
                ↪padding=1),
                nn.InstanceNorm2d(512),
                nn.LeakyReLU(0.2),
            )
            self.convb7 = nn.Sequential(
                nn.Conv2d(in_channels=512, out_channels=512, stride=2, kernel_size=4,
                ↪padding=1),
                nn.InstanceNorm2d(512),
                nn.LeakyReLU(0.2),
            )
            self.convb8 = nn.Sequential(

```

```

        nn.Conv2d(in_channels=512, out_channels=512, stride=2, kernel_size=4,
↪padding=1),
        nn.LeakyReLU(0.2),
    )

    self.conv1b1 = nn.Sequential(
        nn.ConvTranspose2d(in_channels=512, out_channels=512, kernel_size=4,
↪stride=2, padding=1),
        nn.InstanceNorm2d(512),
        nn.Dropout(0.5),
        nn.ReLU(),
    )
    self.conv1b2 = nn.Sequential(
        nn.ConvTranspose2d(in_channels=1024, out_channels=512, kernel_size=4,
↪stride=2, padding=1),
        nn.InstanceNorm2d(512),
        nn.Dropout(0.5),
        nn.ReLU(),
    )
    self.conv1b3 = nn.Sequential(
        nn.ConvTranspose2d(in_channels=1024, out_channels=512, kernel_size=4,
↪stride=2, padding=1),
        nn.InstanceNorm2d(512),
        nn.Dropout(0.5),
        nn.ReLU(),
    )
    self.conv1b4 = nn.Sequential(
        nn.ConvTranspose2d(in_channels=1024, out_channels=512, kernel_size=4,
↪stride=2, padding=1),
        nn.InstanceNorm2d(512),
        nn.Dropout(0.5),
        nn.ReLU(),
    )
    self.conv1b5 = nn.Sequential(
        nn.ConvTranspose2d(in_channels=1024, out_channels=256, kernel_size=4,
↪stride=2, padding=1),
        nn.InstanceNorm2d(256),
        nn.Dropout(0.5),
        nn.ReLU(),
    )
    self.conv1b6 = nn.Sequential(
        nn.ConvTranspose2d(in_channels=512, out_channels=128, kernel_size=4,
↪stride=2, padding=1),
        nn.InstanceNorm2d(128),
        nn.Dropout(0.5),
        nn.ReLU(),
    )

```

```

    )
    self.conv7b = nn.Sequential(
        nn.ConvTranspose2d(in_channels=256, out_channels=64, kernel_size=4,
↪stride=2, padding=1),
        nn.InstanceNorm2d(64),
        nn.Dropout(0.5),
        nn.ReLU(),
    )
    self.conv8b = nn.Sequential(
        nn.ConvTranspose2d(in_channels=128, out_channels=3, kernel_size=4,
↪stride=2, padding=1),
    )

    def forward(self, x):
        feature1 = self.conv1b(x)
        feature2 = self.conv2b(feature1)
        feature3 = self.conv3b(feature2)
        feature4 = self.conv4b(feature3)
        feature5 = self.conv5b(feature4)
        feature6 = self.conv6b(feature5)
        feature7 = self.conv7b(feature6)
        feature8 = self.conv8b(feature7)

        ufeature1 = self.conv9b(feature8)
        ufeature1c = torch.concat([ufeature1, feature7], dim=1)
        ufeature2 = self.conv10b(ufeature1c)
        ufeature2c = torch.concat([ufeature2, feature6], dim=1)
        ufeature3 = self.conv11b(ufeature2c)
        ufeature3c = torch.concat([ufeature3, feature5], dim=1)
        ufeature4 = self.conv12b(ufeature3c)
        ufeature4c = torch.concat([ufeature4, feature4], dim=1)
        ufeature5 = self.conv13b(ufeature4c)
        ufeature5c = torch.concat([ufeature5, feature3], dim=1)
        ufeature6 = self.conv14b(ufeature5c)
        ufeature6c = torch.concat([ufeature6, feature2], dim=1)
        ufeature7 = self.conv15b(ufeature6c)
        ufeature7c = torch.concat([ufeature7, feature1], dim=1)
        ufeature8 = self.conv16b(ufeature7c)
        output = F.tanh(ufeature8)

    return output

```

```
generator = Generator().cuda()
```

```

[21]: #TODO14 verify the generator
from torchsummary import summary

```

```
summary(generator, input_size=(3, 256, 256))
```

Layer (type)	Output Shape	Param #
Conv2d-1	[-1, 64, 128, 128]	3,136
LeakyReLU-2	[-1, 64, 128, 128]	0
Conv2d-3	[-1, 128, 64, 64]	131,200
InstanceNorm2d-4	[-1, 128, 64, 64]	0
LeakyReLU-5	[-1, 128, 64, 64]	0
Conv2d-6	[-1, 256, 32, 32]	524,544
InstanceNorm2d-7	[-1, 256, 32, 32]	0
LeakyReLU-8	[-1, 256, 32, 32]	0
Conv2d-9	[-1, 512, 16, 16]	2,097,664
InstanceNorm2d-10	[-1, 512, 16, 16]	0
LeakyReLU-11	[-1, 512, 16, 16]	0
Conv2d-12	[-1, 512, 8, 8]	4,194,816
InstanceNorm2d-13	[-1, 512, 8, 8]	0
LeakyReLU-14	[-1, 512, 8, 8]	0
Conv2d-15	[-1, 512, 4, 4]	4,194,816
InstanceNorm2d-16	[-1, 512, 4, 4]	0
LeakyReLU-17	[-1, 512, 4, 4]	0
Conv2d-18	[-1, 512, 2, 2]	4,194,816
InstanceNorm2d-19	[-1, 512, 2, 2]	0
LeakyReLU-20	[-1, 512, 2, 2]	0
Conv2d-21	[-1, 512, 1, 1]	4,194,816
LeakyReLU-22	[-1, 512, 1, 1]	0
ConvTranspose2d-23	[-1, 512, 2, 2]	4,194,816
InstanceNorm2d-24	[-1, 512, 2, 2]	0
Dropout-25	[-1, 512, 2, 2]	0
ReLU-26	[-1, 512, 2, 2]	0
ConvTranspose2d-27	[-1, 512, 4, 4]	8,389,120
InstanceNorm2d-28	[-1, 512, 4, 4]	0
Dropout-29	[-1, 512, 4, 4]	0
ReLU-30	[-1, 512, 4, 4]	0
ConvTranspose2d-31	[-1, 512, 8, 8]	8,389,120
InstanceNorm2d-32	[-1, 512, 8, 8]	0
Dropout-33	[-1, 512, 8, 8]	0
ReLU-34	[-1, 512, 8, 8]	0
ConvTranspose2d-35	[-1, 512, 16, 16]	8,389,120
InstanceNorm2d-36	[-1, 512, 16, 16]	0
Dropout-37	[-1, 512, 16, 16]	0
ReLU-38	[-1, 512, 16, 16]	0
ConvTranspose2d-39	[-1, 256, 32, 32]	4,194,560
InstanceNorm2d-40	[-1, 256, 32, 32]	0
Dropout-41	[-1, 256, 32, 32]	0
ReLU-42	[-1, 256, 32, 32]	0
ConvTranspose2d-43	[-1, 128, 64, 64]	1,048,704

InstanceNorm2d-44	[-1, 128, 64, 64]	0
Dropout-45	[-1, 128, 64, 64]	0
ReLU-46	[-1, 128, 64, 64]	0
ConvTranspose2d-47	[-1, 64, 128, 128]	262,208
InstanceNorm2d-48	[-1, 64, 128, 128]	0
Dropout-49	[-1, 64, 128, 128]	0
ReLU-50	[-1, 64, 128, 128]	0
ConvTranspose2d-51	[-1, 3, 256, 256]	6,147

=====

Total params: 54,409,603
Trainable params: 54,409,603
Non-trainable params: 0

Input size (MB): 0.75
Forward/backward pass size (MB): 100.80
Params size (MB): 207.56
Estimated Total Size (MB): 309.11

3.8 Data preparation

After the model is initialized, we then create a dataloader to sample the training data. In this paper, to sample the training data, you have to sequentially perform the following steps :

1. Randomly sample the data from the training set
2. Resizing both input and target to 286×286 .
3. Randomly cropping back both images to size 256×256 .
4. Random mirroring the images

TODO15: Implement a dataloader based on the description above. You are allowed to use the function in `torchvision.transforms` (<https://pytorch.org/vision/main/transforms.html>).

```
[22]: # TODO15 implement a dataloader
from torch.utils.data import Dataset, DataLoader
from torchvision import transforms

BATCH_SIZE = 1

class FacadesDataset(Dataset):
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __len__(self):
        return len(self.x)

    def __getitem__(self, idx):
        input_img = torch.from_numpy(self.x[idx])
        target_img = torch.from_numpy(self.y[idx])
```

```

        # if we don't concat then the random transformation won't be the same
        ↪ for both of the image
        combined = torch.cat([input_img, target_img], dim=0) # concat with
        ↪ channel dim (6, w, h)

        combined = transforms.functional.resize(combined, size=[286, 286],
        ↪ antialias=True)
        combined = transforms.RandomResizedCrop(size=(256, 256))(combined)
        combined = transforms.RandomHorizontalFlip(p=0.5)(combined)

        input_aug = combined[:3, :, :]
        target_aug = combined[3:, :, :]

        return input_aug, target_aug

train_dataset = FacadesDataset(trainX, trainY)
val_dataset = FacadesDataset(valX, valY)

train_loader = DataLoader(train_dataset, shuffle=True, batch_size=BATCH_SIZE)
val_loader = DataLoader(val_dataset, shuffle=False, batch_size=BATCH_SIZE)

```

3.8.1 Dataloader verification

TODO16: Show that the implemented dataloader is working as intended. For instance, are both input and output are flipped and cropped correctly? To obtain a full score, you have to show at least eight data points.

```

[23]: #TODO16 show that the dataloader is working properly
import matplotlib.pyplot as plt
import numpy as np

samples = []
data_iter = iter(train_loader)

for i in range(8):
    samples.append(next(data_iter))

fig, axes = plt.subplots(4, 4, figsize=(16, 16))
plt.subplots_adjust(wspace=0.1, hspace=0.3)

for i in range(8):
    input_tensor, target_tensor = samples[i]

    # need to [0] because first dimension is batch
    # permute channel to last dimension

```

```

img_input = input_tensor[0].permute(1, 2, 0).cpu().numpy()
img_target = target_tensor[0].permute(1, 2, 0).cpu().numpy()

# denormalize
img_input = (img_input * 0.5) + 0.5
img_target = (img_target * 0.5) + 0.5

row = i // 2
col_start = (i % 2) * 2

axes[row, col_start].imshow(img_input)
axes[row, col_start].set_title(f"Sample {i+1}: Input")
axes[row, col_start].axis('off')

axes[row, col_start+1].imshow(img_target)
axes[row, col_start+1].set_title(f"Sample {i+1}: Target")
axes[row, col_start+1].axis('off')

plt.show()

```

3.9 Parameter Initialization

Model hyperparameters and optimizers have already been prepared.

```
[24]: import torch.optim as optim
      from tqdm import tqdm
      lr = 2e-4
      LAMBDA = 100
      G_optimizer = optim.Adam(generator.parameters(), lr=lr, betas = (0.5, 0.999))
      D_optimizer = optim.Adam(discriminator.parameters(), lr=lr, betas = (0.5, 0.
      ↪999))
```

```
device = "cuda" if torch.cuda.is_available() else "cpu"
```

3.10 Training loop

The training process has the following specific requirements: - The objective is divided by 2 while optimizing D , which slows down the rate at which D learns relative to G . - This paper trains the generator G to maximize $\log D(x, G(x, z))$ instead of minimizing $\log(1 - D(x, G(x, z)))$ as the latter term does not provide sufficient gradient.

TODO17: Sample the data using the dataloader.

TODO18: Calculate the discriminator loss and update the discriminator. - During the update, the loss term $\log(1 - D(x, G(x, z)))$ contains both generator and discriminator. However, we only want to update the discriminator. Therefore, you have to detach the input from the generator graph first. Read <https://pytorch.org/docs/stable/generated/torch.Tensor.detach.html> for additional detail.

TODO19: Calculate the generator loss and update the generator.

HINT

Hint 1: If you are struggling with this part, you could also read the PyTorch DCGAN tutorial as a guideline (https://pytorch.org/tutorials/beginner/dcgan_faces_tutorial.html).

Hint 2: You could remove the L1 loss while debugging since the generator G could still generate the synthetic image even if the L1 loss is removed, though at the cost of increasing image artifacts.

Note

The training schedule in this homework is only one eighth of the original schedule. It is expected that the generated image quality is worse than the one shown in the paper. Nevertheless, the generated facade should still resemble an actual one.

```
[25]: losses = {'D' : [None], 'G' : [None]}

train_it = iter(train_loader)

bce_fn = nn.BCELoss(reduction='mean')
l1_fn = nn.L1Loss()

for i in tqdm(range(20001)):
    #TODO17 sample the data from the dataloader
    try:
        input_img, target_img = next(train_it)
    except:
        train_it = iter(train_loader)
        input_img, target_img = next(train_it)

    input_img, target_img = input_img.to(device), target_img.to(device)

    generated_img = generator(input_img)
```

```

fake_predict = discriminator(input_img, generated_img.detach()) # prevent
↳weight to update to discriminator
real_predict = discriminator(input_img, target_img)

#TODO18 calculate the discriminator loss and update the discriminator
D_optimizer.zero_grad()

fake_label = torch.zeros_like(fake_predict) # label of fake one is zero
fake_loss = bce_fn(fake_predict, fake_label)

real_label = torch.ones_like(real_predict)
real_loss = bce_fn(real_predict, real_label)

D_loss = (fake_loss + real_loss) / 2
D_loss.backward()
losses['D'].append(D_loss.item())

D_optimizer.step()

#TODO19 calculate the generator loss and update the generator
G_optimizer.zero_grad()

fake_predict_for_g = discriminator(input_img, generated_img)
target_real = torch.ones_like(fake_predict_for_g) # target for G is 1 because
↳G wants discriminator to answer 1
loss_gan = bce_fn(fake_predict_for_g, target_real)

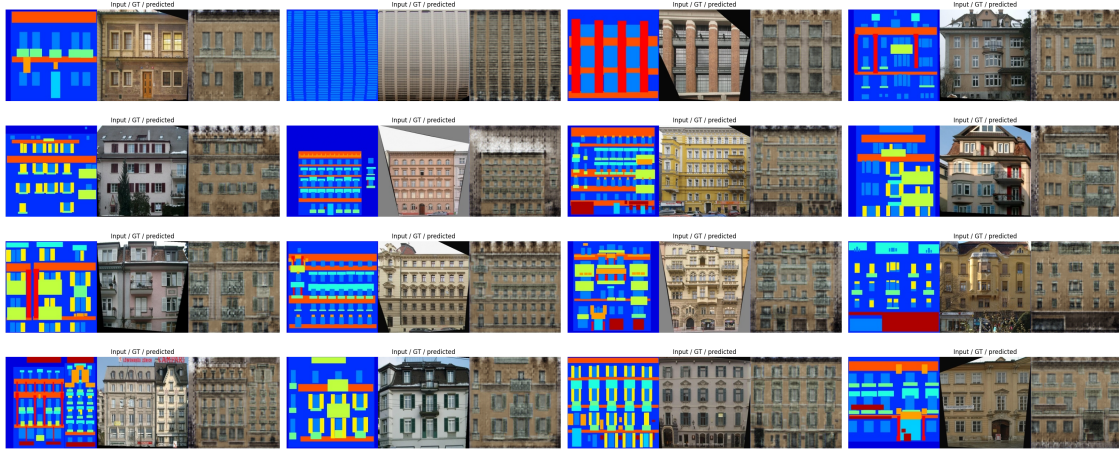
g_loss = loss_gan + LAMBDA * l1_fn(generated_img, target_img)
g_loss.backward()
losses['G'].append(g_loss.item())

G_optimizer.step()
# Output visualization : If your reimplementation is correct, the generated
↳images should start resembling a facade after 2,500 iterations
if(i % 2500 == 0):
    with torch.no_grad():
        print(losses['D'][-1], losses['G'][-1])
        plt.figure(figsize = (40, 16))
        gs1 = gridspec.GridSpec(4, 4)
        gs1.update(wspace=0.025)

        sampleX_vis = 0.5 * valX[:16][:, ::-1, :, :] + 0.5
        sampleY = 0.5 * valY[:16][:, ::-1, :, :] + 0.5
        sampleX = torch.tensor(valX[:16]).cuda()
        pred_val = 0.5 * generator(sampleX).cpu().detach().numpy()[:, ::-1, :, :]
↳+ 0.5
        vis = np.concatenate([sampleX_vis, sampleY, pred_val], axis = 3)

```

```
for i in range(vis.shape[0]):  
    ax1 = plt.subplot(gs1[i])  
    plt.title('Input / GT / predicted')  
    plt.axis('off')  
    plt.imshow( vis[i].transpose(1, 2, 0) )  
plt.show()
```



100%| | 20001/20001 [10:34<00:00, 31.54it/s]

3.11 (Optional)

Combine the WGAN-GP loss with the pix2pix objective.

$\text{Loss}_G = -\text{Mean}(D(\text{fake})) + \lambda * L1(\text{fake}, \text{real})$

$\text{Loss}_D = \text{Mean}(D(\text{fake})) - \text{Mean}(D(\text{real})) + 10 *$

```
[26]: import torch.optim as optim
      from tqdm import tqdm
      lr = 2e-4
      LAMBDA_G = 100
      LAMBDA_GP = 10
      G_optimizer = optim.Adam(generator.parameters(), lr=lr, betas = (0.0, 0.9))
      D_optimizer = optim.Adam(discriminator.parameters(), lr=lr, betas = (0.0, 0.9))

      device = "cuda" if torch.cuda.is_available() else "cpu"
```

```
[27]: losses = {'D' : [None], 'G' : [None]}

      train_it = iter(train_loader)

      bce_fn = nn.BCELoss(reduction='mean')
      l1_fn = nn.L1Loss()

      for i in tqdm(range(30001)):
          # dataloader
          try:
              input_img, target_img = next(train_it)
          except:
              train_it = iter(train_loader)
```

```

    input_img, target_img = next(train_it)

input_img, target_img = input_img.to(device), target_img.to(device)

generated_img = generator(input_img)

# discriminator
D_optimizer.zero_grad()

fake_d = discriminator(input_img, generated_img.detach()).mean()
real_d = discriminator(input_img, target_img).mean()

alpha = torch.rand((target_img.shape[0], 1, 1, 1)).to(device)
interpolates = (alpha * target_img + ((1 - alpha) * generated_img.detach())).
↳requires_grad_(True)
d_interpolates = discriminator(input_img, interpolates)
fake = torch.ones(d_interpolates.shape).to(device)

gradients = torch.autograd.grad(
    outputs=d_interpolates,
    inputs=interpolates,
    grad_outputs=fake,
    create_graph=True,
    retain_graph=True,
    only_inputs=True,
)[0]

gradients = gradients.view(gradients.size(0), -1)
gradient_penalty = ((gradients.norm(2, dim=1) - 1) ** 2).mean()

D_loss = fake_d - real_d + LAMBDA_GP * gradient_penalty
D_loss.backward()
losses['D'].append(D_loss.item())

D_optimizer.step()

# generator
G_optimizer.zero_grad()

loss_gan = - discriminator(input_img, generated_img).mean()

g_loss = loss_gan + LAMBDA_G * l1_fn(generated_img, target_img)
g_loss.backward()
losses['G'].append(g_loss.item())

G_optimizer.step()

```

```

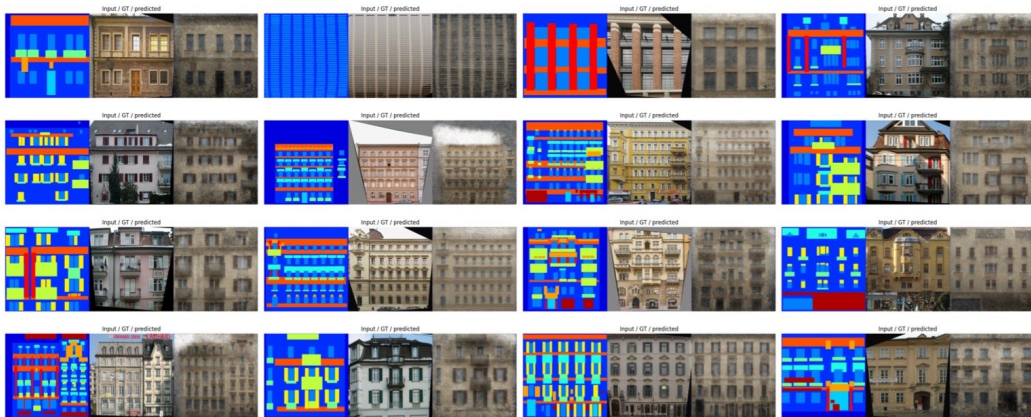
# Output visualization : If your reimplementation is correct, the generated
↪ images should start resembling a facade after 2,500 iterations
if(i % 2500 == 0):
    with torch.no_grad():
        print(losses['D'][-1], losses['G'][-1])
        plt.figure(figsize = (40, 16))
        gs1 = gridspec.GridSpec(4, 4)
        gs1.update(wspace=0.025)

        sampleX_vis = 0.5 * valX[:16][:, ::-1, :, :] + 0.5
        sampleY = 0.5 * valY[:16][:, ::-1, :, :] + 0.5
        sampleX = torch.tensor(valX[:16]).cuda()
        pred_val = 0.5 * generator(sampleX).cpu().detach().numpy()[:, ::-1, :, :]
↪ + 0.5

        vis = np.concatenate([sampleX_vis, sampleY, pred_val], axis = 3)
        for i in range(vis.shape[0]):
            ax1 = plt.subplot(gs1[i])
            plt.title('Input / GT / predicted')
            plt.axis('off')
            plt.imshow( vis[i].transpose(1, 2, 0) )
        plt.show()

```

0.04514505714178085 40.93036651611328



100%| | 30001/30001 [23:41<00:00, 21.10it/s]